# Algorithmics and Data Structures

## Session 2
## Data Structures

**Ali Ismail-Fawaz**

ENSISA, Université Haute-Alsace

# Overview

- **Goal**:
  - Understand what data structures are
  - Learn why they are essential in computer science and programming
  - Get an overview of common types of data structures
  - Choose which one to use with respect to the application

- **What is a Data Structure ?**
  - A data structure is a way to organize, store, and access data efficiently
  - It defines the relationship between data elements and how operations can be performed on them

- **Why Use Data Structures ?**
  - To handle data efficiently in terms of time and space
  - To solve complex problems effectively
  - To adapt the data representation to the need of the problem

# Contents

Primitive Data Types

Arrays

Lists

Stacks

Queues

Hash Tables

Structures in C

# Primitive Data Types

- **What are Primitive Types ?**
  - Basic, built-in data types provided by the programming language
  - They form the foundation for more complex data structures

- **Common Primitive Types**:
  - `int`, `long`: Integer numbers (ex: 42)
  - `float`, `double`: Real numbers (ex: 3.14)
  - `char`: Single characters (ex: 'A')
  - `Bool`: Boolean (true/false)

# Primitive Data Types

- **What are Primitive Types ?**
  - Basic, built-in data types provided by the programming language
  - They form the foundation for more complex data structures
- **Common Primitive Types**:
  - `int`, `long`: Integer numbers (ex: 42)
  - `float`, `double`: Real numbers (ex: 3.14)
  - `char`: Single characters (ex: 'A')
  - `Bool`: Boolean (true/false)
- **Why They're Limited**:
  - Cannot organize or link complex sets of data
    - Example: Managing a list of students along with their grades
  - Inefficient for certain operations on large data sets
    - Poor performance in fast search or structured access
    - Unable to model hierarchical or interconnected relationships

# Primitive Data Types

- **What are Primitive Types ?**
  - Basic, built-in data types provided by the programming language
  - They form the foundation for more complex data structures
- **Common Primitive Types**:
  - `int`, `long`: Integer numbers (ex: 42)
  - `float`, `double`: Real numbers (ex: 3.14)
  - `char`: Single characters (ex: 'A')
  - `Bool`: Boolean (true/false)
- **Why They're Limited**:
  - Cannot organize or link complex sets of data
    - Example: Managing a list of students along with their grades
  - Inefficient for certain operations on large data sets
    - Poor performance in fast search or structured access
    - Unable to model hierarchical or interconnected relationships
- More advanced structures are needed to meet these needs

# C Primitive Data Types Overview

| Type | Size (bytes) | Range (approx) | Format |
|---|---|---|---|
| `short int` | 2 | $-32{,}768$ to $32{,}767$ | %hd |
| `unsigned short int` | 2 | $0$ to $65{,}535$ | %hu |
| `unsigned int` | 4 | $0$ to $4.29 \times 10^9$ | %u |
| `int` | 4 | $-2.14 \times 10^9$ to $2.14 \times 10^9$ | %d |
| `long int` | 8 | $-9.22 \times 10^{18}$ to $+$ | %ld |
| `unsigned long int` | 8 | $0$ to $1.84 \times 10^{19}$ | %lu |
| `long long int` | 8 | same as `long int` | %lld |
| `unsigned long long int` | 8 | $0$ to $1.84 \times 10^{19}$ | %llu |
| `signed char` | 1 | $-128$ to $127$ | %hhd |
| `unsigned char` | 1 | $0$ to $255$ | %hhu |
| `float` | 4 | $\pm 3.4 \times 10^{38}$ | %f |
| `double` | 8 | $\pm 1.7 \times 10^{308}$ | %lf |
| `long double` | 16 | $\pm 1.1 \times 10^{4932}$ | %Lf |

Based on GCC (64-bit Linux, LP64). Actual values may differ by compiler/platform.

*Note: The `_Bool` type (from C99) uses 1 byte and stores only `0` (false) or `1` (true).*

# Contents

Primitive Data Types

Arrays

Lists

Stacks

Queues

Hash Tables

Structures in C

# Arrays

- **Definition:** An array is a fixed-size collection of elements of the same type stored together under one name

# Arrays

- **Definition:** An array is a fixed-size collection of elements of the same type stored together under one name
- **Indexing:** Each element is accessed using an index, starting from 0.
  Example: `a[0]` is the first element, `a[1]` is the second, etc.

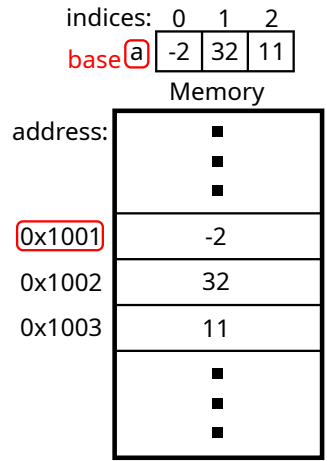
# Arrays

- **Definition:** An array is a fixed-size collection of elements of the same type stored together under one name
- **Indexing:** Each element is accessed using an index, starting from 0.
  Example: `a[0]` is the first element, `a[1]` is the second, etc.
- **Homogeneity:** All elements must be of the same data type
  Example: all `int`, or all `char` etc.

# Arrays

- **Definition:** An array is a fixed-size collection of elements of the same type stored together under one name
- **Indexing:** Each element is accessed using an index, starting from 0.
  Example: `a[0]` is the first element, `a[1]` is the second, etc.
- **Homogeneity:** All elements must be of the same data type
  Example: all `int`, or all `char` etc.
- **Memory Representation:** Elements are stored one after another in memory (no gaps). The address of `a[i]` is computed using the formula:

  $$\texttt{address of } a[i] = \texttt{base address} + i \times \texttt{size of each element}$$

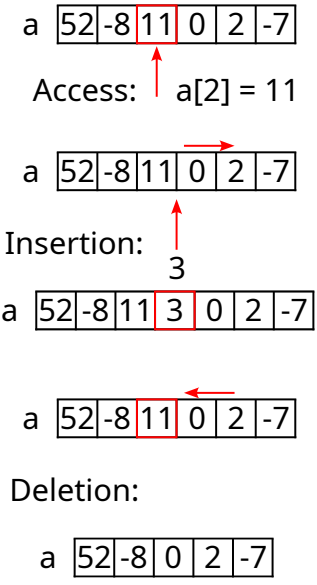


# Operations on Arrays

1. **Direct Access**
   - Access any element using its index: `a[i]`
   - **Time Complexity:** $\mathcal{O}(1)$

2. **Insertion (in the middle)**
   - All elements after the insertion point must be shifted right.
   - **Time Complexity:** $\mathcal{O}(n)$

3. **Deletion (from the middle)**
   - All elements after the deleted item must be shifted left.
   - **Time Complexity:** $\mathcal{O}(n)$



# Why Arrays?

**Advantages:**

- Fast and direct access to elements ($\mathcal{O}(1)$)
- Simple and easy to implement
- Efficient use of memory for fixed-size collections

**Disadvantages:**

- Fixed size, must be defined at initialization
- Not suitable for dynamic resizing
- Costly insertions and deletions in the middle ($\mathcal{O}(n)$)

# Array Algorithms

## Direct Access

Input: array $a$, index $i$
Output: value at position $i$
Instructions:

1. Return $a[i]$

## Deletion from Array

Input: array $a$, size $n$, index $i$
Output: array $a$ without the element at position $i$
Instructions:

1. For $j$ from $i$ to $n-2$, do
   $a[j] \leftarrow a[j+1]$
2. $n \leftarrow n-1$

## Insertion in Array

Input: array $a$, size $n$, index $i$, value $x$
Output: array $a$ with $x$ inserted at position $i$
Instructions:

1. For $j$ from $n-1$ down to $i$, do
   $a[j+1] \leftarrow a[j]$
2. Set $a[i] \leftarrow x$
3. $n \leftarrow n+1$

# Contents

Primitive Data Types

Arrays

Lists

Stacks

Queues

Hash Tables

Structures in C

# Lists

**What is a List?**

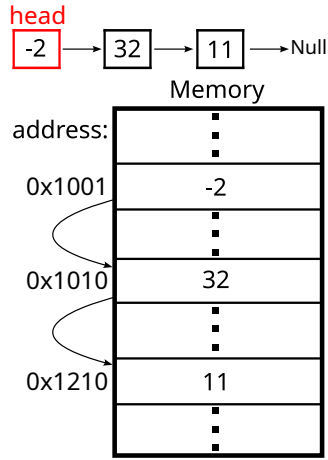
- A list is a dynamic data structure made of nodes
- Each node stores a value and a pointer to the next node
- The list grows and shrinks at runtime, no need to know the size in advance

**List vs Array**

- Arrays: fixed size, contiguous memory, fast random access
- Lists: dynamic size, flexible memory allocation, but slower access by index

**Advantages of Lists**

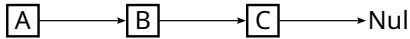
- Easy insertions and deletions at any position ($\mathcal{O}(1)$ if pointer is known). No need to shift elements
- Ideal for when the number of elements is not known



# Types of Linked Lists

**1. Singly Linked List**

- Each node contains a value and a pointer to the next node
- Traversal is only possible in one direction (forward)

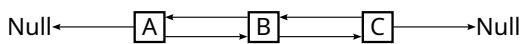


# Types of Linked Lists

1. **Singly Linked List**
   - Each node contains a value and a pointer to the next node
   - Traversal is only possible in one direction (forward)



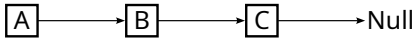

2. **Doubly Linked List**
   - Each node contains a value, a pointer to the next node, and a pointer to the previous node
   - Supports forward and backward traversal



# Types of Linked Lists

## 1. Singly Linked List

- Each node contains a value and a pointer to the next node
- Traversal is only possible in one direction (forward)



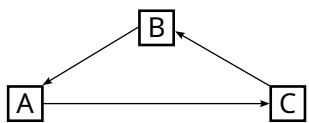

## 2. Doubly Linked List

- Each node contains a value, a pointer to the next node, and a pointer to the previous node
- Supports forward and backward traversal




## 3. Circular Linked List

- The last node links back to the first node
- No clear end, traversal stops when reaching the starting point again



# Comparison of List Types

| List Type | Advantages | Disadvantages |
|---|---|---|
| Singly | Simple, light | No backward traversal |
| Doubly | Two-way traversal | More memory, complex |
| Circular | No end, good for loops | Hard to stop safely |

| Operation | Singly | Doubly | Circular |
|---|---|---|---|
| Access (head/tail) | $\mathcal{O}(1)/\mathcal{O}(n)$ | $\mathcal{O}(1)/\mathcal{O}(1)$ | $\mathcal{O}(1)/\mathcal{O}(n)$ |
| Search | $\mathcal{O}(n)$ | $\mathcal{O}(n)$ | $\mathcal{O}(n)$ |
| Insertion (head/tail) | $\mathcal{O}(1)/\mathcal{O}(n)$ | $\mathcal{O}(1)/\mathcal{O}(1)$ | $\mathcal{O}(1)/\mathcal{O}(n)$ |
| Deletion (head/tail) | $\mathcal{O}(1)/\mathcal{O}(n)$ | $\mathcal{O}(1)/\mathcal{O}(1)$ | $\mathcal{O}(1)/\mathcal{O}(n)$ |
| Read | $\mathcal{O}(n)$ | $\mathcal{O}(n)$ | $\mathcal{O}(n)$ |
| Memory / Node | 1 pointer | 2 pointers | 1 pointer |

# Contents

Primitive Data Types

Arrays

Lists

Stacks

Queues

Hash Tables

Structures in C

# Stacks

**Concept**:

- Stacks follow the rule of **LIFO** (**L**ast **I**n **F**irst **O**ut)
- Metaphor using stack of plates: you add and remove items only from the top, following the LIFO rule

**Operations**:

- **Push**: Add an element to the top of the stack
- **Pop**: Remove an element from the top of the stack
- **Peek**: Return the element at the top of the stack without removing it from the stack
- All three operations have a time complexity of $\mathcal{O}(1)$

# Stack Operations

## Push

Input: array $S$, max size $N$, curr size $n$, value $x$
Output: array with $x$ added on top
Instructions:

1. $S[n-1] \leftarrow x$
2. $n \leftarrow n+1$

## Peek

Input: array $S$, max size $N$, curr size $n$
Output: value at the top
Instructions:

1. Return $S[n-1]$

## Pop

Input: array $S$, max size $N$, curr size $n$
Output: value removed from top
Instructions:

1. $x \leftarrow S[n-1]$
2. $n \leftarrow n-1$
3. Return $x$

# Advantages and Disadvantages of Stacks

**Advantages:**

- **Simplicity**: Easy to implement and understand
- **Performance**: Push, peek and pop operations are fast ($\mathcal{O}(1)$)
- **Versatile Applications**:
  - Function call management (recursion)
  - Undo/redo systems (text editors, IDEs)
  - Expression evaluation (parsers, calculators)

**Disadvantages:**

- **Limited Access**: Only the top element is accessible
- **Capacity May Be Limited**: Fixed in arrays; dynamic in linked lists
- **Not Suited for Complex Searches**: time complexity of a search $\mathcal{O}(n)$

# Call Stack of a Simple Recursive Function

## C Code: Countdown

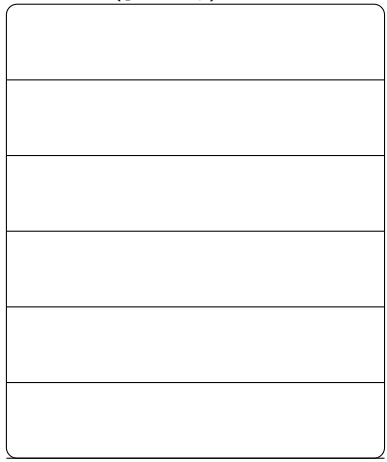
```c
#include <stdio.h>

void countdown(int n) {
    if (n == 0) {
        printf("END!");
    } else {
        printf("%d", n);
        countdown(n - 1);
    }
}

int main() {
    countdown(3);
    return 0;
}
```

## Output

## Call Stack (grows up)



# Call Stack of a Simple Recursive Function

## C Code: Countdown

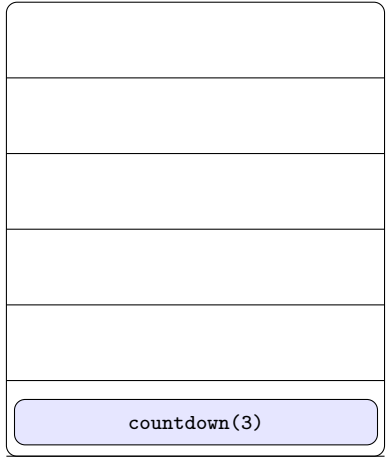
```c
#include <stdio.h>

void countdown(int n) {
    if (n == 0) {
        printf("END!");
    } else {
        printf("%d", n);
        countdown(n - 1);
    }
}

int main() {
    countdown(3);←
    return 0;
}
```

## Output

## Call Stack (grows up)



# Call Stack of a Simple Recursive Function

## C Code: Countdown

```c
#include <stdio.h>

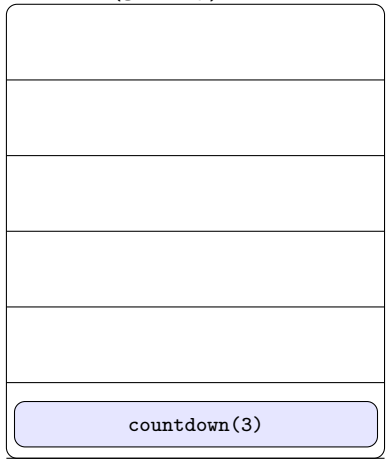
void countdown(int n) {
    if (n == 0) {
        printf("END!");
    } else {
        printf("%d", n);←
        countdown(n - 1);
    }
}

int main() {
    countdown(3);
    return 0;
}
```

## Output

3

## Call Stack (grows up)



# Call Stack of a Simple Recursive Function

## C Code: Countdown

```c
#include <stdio.h>

void countdown(int n) {
    if (n == 0) {
        printf("END!");
    } else {
        printf("%d", n);
        countdown(n - 1);←
    }
}

int main() {
    countdown(3);
    return 0;
}
```

## Output

3

## Call Stack (grows up)



# Call Stack of a Simple Recursive Function

## C Code: Countdown

```c
#include <stdio.h>

void countdown(int n) {
    if (n == 0) {
        printf("END!");
    } else {
        printf("%d", n);←
        countdown(n - 1);
    }
}

int main() {
    countdown(3);
    return 0;
}
```

## Output

32

## Call Stack (grows up)



# Call Stack of a Simple Recursive Function

## C Code: Countdown

```c
#include <stdio.h>

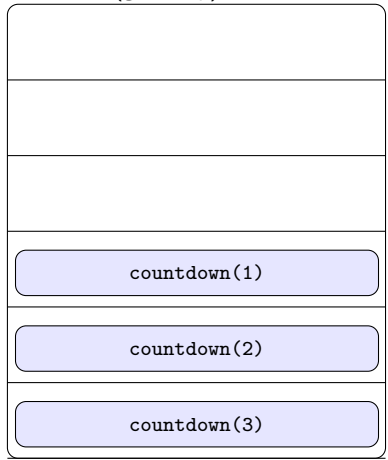
void countdown(int n) {
    if (n == 0) {
        printf("END!");
    } else {
        printf("%d", n);
        countdown(n - 1);←
    }
}

int main() {
    countdown(3);
    return 0;
}
```

## Output

32

## Call Stack (grows up)



# Call Stack of a Simple Recursive Function

## C Code: Countdown

```c
#include <stdio.h>

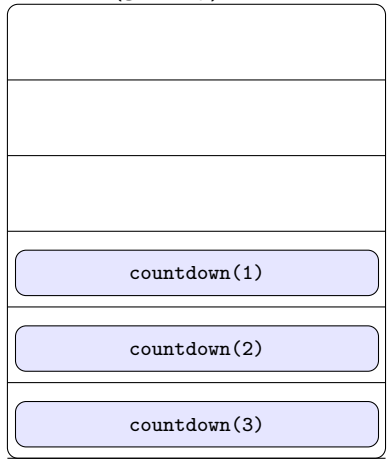
void countdown(int n) {
    if (n == 0) {
        printf("END!");
    } else {
        printf("%d", n);←
        countdown(n - 1);
    }
}

int main() {
    countdown(3);
    return 0;
}
```

## Output

```
321
```

**Call Stack (grows up)**



# Call Stack of a Simple Recursive Function

## C Code: Countdown

```c
#include <stdio.h>

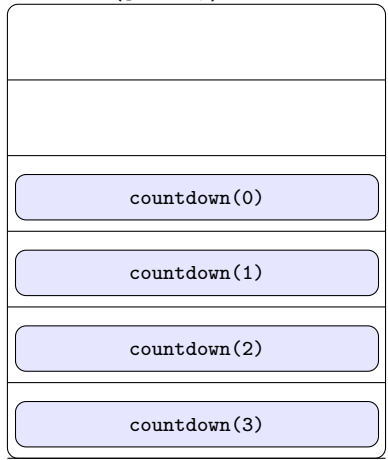
void countdown(int n) {
    if (n == 0) {
        printf("END!");
    } else {
        printf("%d", n);
        countdown(n - 1);←
    }
}

int main() {
    countdown(3);
    return 0;
}
```

## Output

321

**Call Stack (grows up)**



# Call Stack of a Simple Recursive Function

## C Code: Countdown

```c
#include <stdio.h>

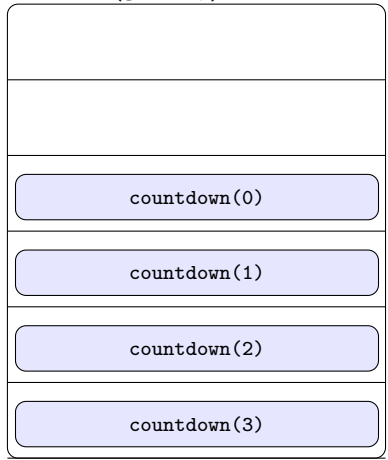
void countdown(int n) {
    if (n == 0) {
        printf("END!");←
    } else {
        printf("%d", n);
        countdown(n - 1);
    }
}

int main() {
    countdown(3);
    return 0;
}
```

## Output

321END!

**Call Stack (grows up)**



# Call Stack of a Simple Recursive Function

## C Code: Countdown

```c
#include <stdio.h>

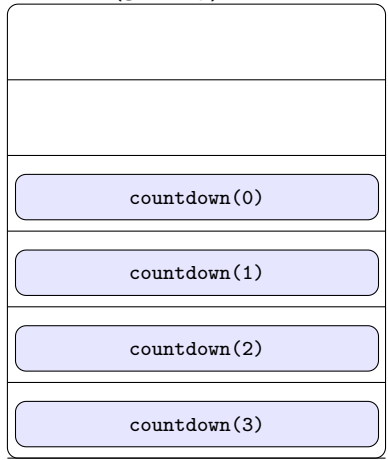
void countdown(int n) {
    if (n == 0) {
        printf("END!");
    } else {
        printf("%d", n);
        countdown(n - 1);
    }
}

int main() {
    countdown(3);
    return 0;
}
```

## Output

321END!

## Call Stack (grows up)



# Call Stack of a Simple Recursive Function

## C Code: Countdown

```c
#include <stdio.h>

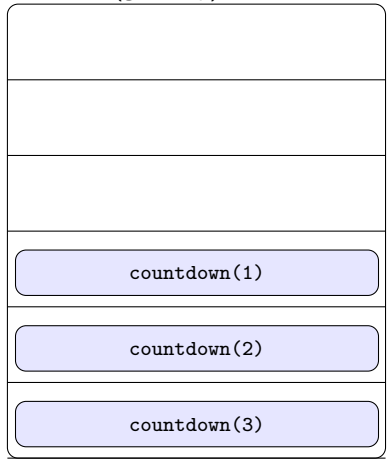
void countdown(int n) {
    if (n == 0) {
        printf("END!");
    } else {
        printf("%d", n);
        countdown(n - 1);
    }
}

int main() {
    countdown(3);
    return 0;
}
```

## Output

321END!

## Call Stack (grows up)



# Call Stack of a Simple Recursive Function

## C Code: Countdown

```c
#include <stdio.h>

void countdown(int n) {
    if (n == 0) {
        printf("END!");
    } else {
        printf("%d", n);
        countdown(n - 1);
    }
}

int main() {
    countdown(3);
    return 0;
}
```

## Output

321END!

## Call Stack (grows up)



# Call Stack of a Simple Recursive Function

## C Code: Countdown

```c
#include <stdio.h>

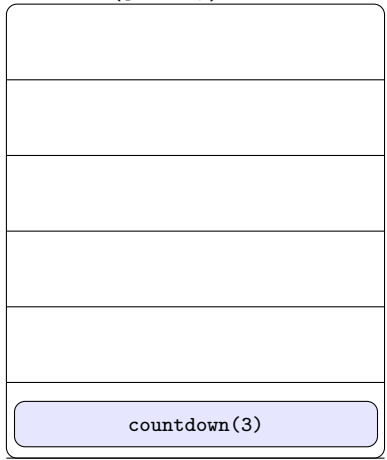
void countdown(int n) {
    if (n == 0) {
        printf("END!");
    } else {
        printf("%d", n);
        countdown(n - 1);
    }
}

int main() {
    countdown(3);
    return 0;
}
```

## Output

321END!

**Call Stack (grows up)**



# Call Stack of a Simple Recursive Function

## C Code: Countdown

```c
#include <stdio.h>

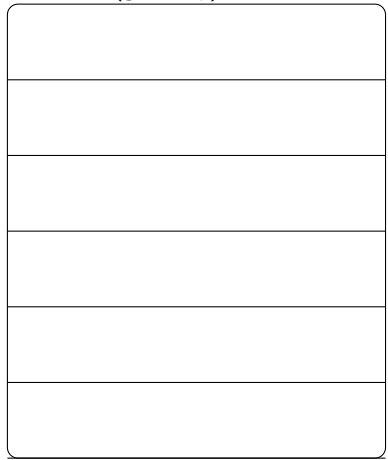
void countdown(int n) {
    if (n == 0) {
        printf("END!");
    } else {
        printf("%d", n);
        countdown(n - 1);
    }
}

int main() {
    countdown(3);
    return 0;
}
```

## Output

321END!

**Call Stack (grows up)**



# Call Stack of a Simple Recursive Function (example 2)

## C Code: Countdown

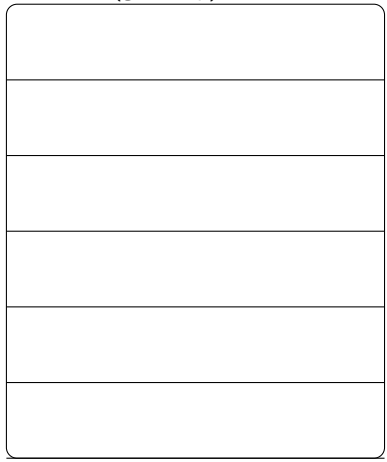
```c
#include <stdio.h>

void countdown(int n) {
    if (n == 0) {
        printf("END!");
    } else {
        countdown(n - 1);
        printf("%d", n);
    }
}

int main() {
    countdown(3);
    return 0;
}
```

## Output

## Call Stack (grows up)



# Call Stack of a Simple Recursive Function (example 2)

## C Code: Countdown

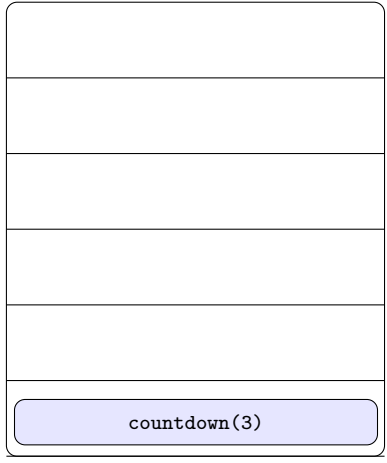
```c
#include <stdio.h>

void countdown(int n) {
    if (n == 0) {
        printf("END!");
    } else {
        countdown(n - 1);
        printf("%d", n);
    }
}

int main() {
    countdown(3);←
    return 0;
}
```

## Output

## Call Stack (grows up)



# Call Stack of a Simple Recursive Function (example 2)

## C Code: Countdown

```c
#include <stdio.h>

void countdown(int n) {
    if (n == 0) {
        printf("END!");
    } else {
        countdown(n - 1);
        printf("%d", n);
    }
}

int main() {
    countdown(3);
    return 0;
}
```

## Output

## Call Stack (grows up)



# Call Stack of a Simple Recursive Function (example 2)

## C Code: Countdown

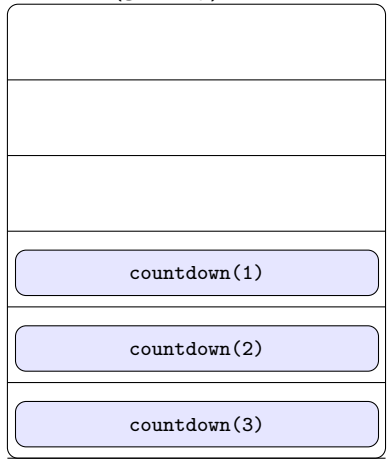
```c
#include <stdio.h>

void countdown(int n) {
    if (n == 0) {
        printf("END!");
    } else {
        countdown(n - 1);
        printf("%d", n);
    }
}

int main() {
    countdown(3);
    return 0;
}
```

## Output

## Call Stack (grows up)



# Call Stack of a Simple Recursive Function (example 2)

## C Code: Countdown

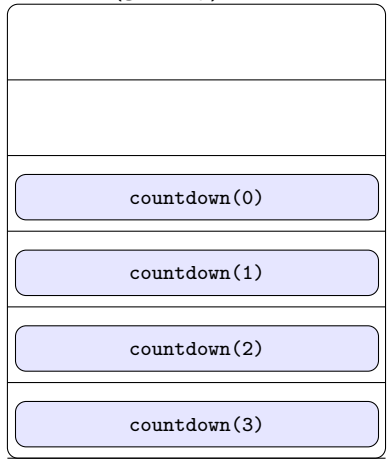
```c
#include <stdio.h>

void countdown(int n) {
    if (n == 0) {
        printf("END!");
    } else {
        countdown(n - 1);
        printf("%d", n);
    }
}

int main() {
    countdown(3);
    return 0;
}
```

## Output

## Call Stack (grows up)



# Call Stack of a Simple Recursive Function (example 2)

## C Code: Countdown

```c
#include <stdio.h>

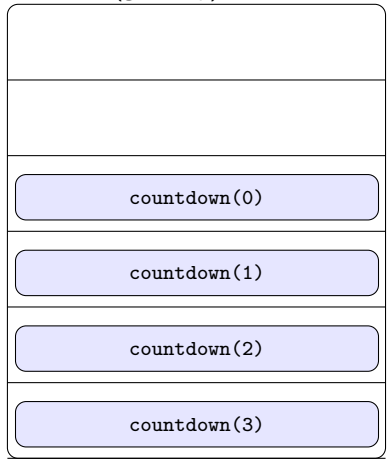
void countdown(int n) {
    if (n == 0) {
        printf("END!");←
    } else {
        countdown(n - 1);
        printf("%d", n);
    }
}

int main() {
    countdown(3);
    return 0;
}
```

## Output

END!

## Call Stack (grows up)



# Call Stack of a Simple Recursive Function (example 2)

## C Code: Countdown

```c
#include <stdio.h>

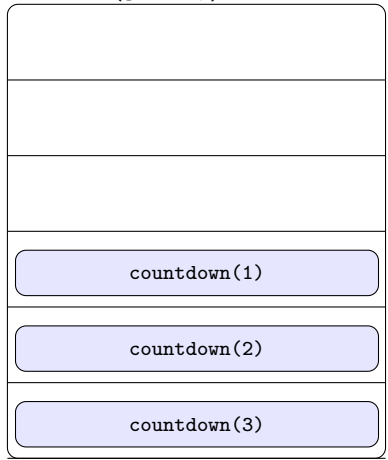
void countdown(int n) {
    if (n == 0) {
        printf("END!");
    } else {
        countdown(n - 1);
        printf("%d", n);←
    }
}

int main() {
    countdown(3);
    return 0;
}
```

## Output

END!1

## Call Stack (grows up)



# Call Stack of a Simple Recursive Function (example 2)

## C Code: Countdown

```c
#include <stdio.h>

void countdown(int n) {
    if (n == 0) {
        printf("END!");
    } else {
        countdown(n - 1);
        printf("%d", n);←
    }
}

int main() {
    countdown(3);
    return 0;
}
```

## Output

END!12

## Call Stack (grows up)



# Call Stack of a Simple Recursive Function (example 2)

## C Code: Countdown

```c
#include <stdio.h>

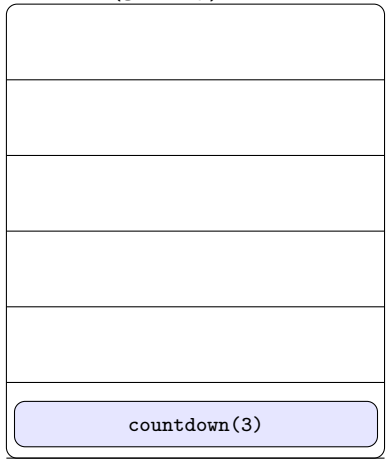
void countdown(int n) {
    if (n == 0) {
        printf("END!");
    } else {
        countdown(n - 1);
        printf("%d", n);←
    }
}

int main() {
    countdown(3);
    return 0;
}
```

## Output

END!123

## Call Stack (grows up)



# Call Stack of a Simple Recursive Function (example 2)

## C Code: Countdown

```c
#include <stdio.h>

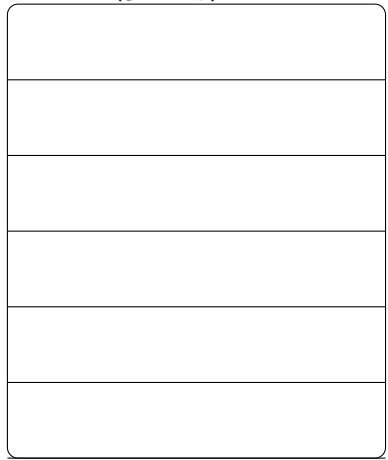
void countdown(int n) {
    if (n == 0) {
        printf("END!");
    } else {
        countdown(n - 1);
        printf("%d", n);
    }
}

int main() {
    countdown(3);
    return 0;
}
```

## Output

END!123

## Call Stack (grows up)



# Contents

Primitive Data Types

Arrays

Lists

Stacks

Queues

Hash Tables

Structures in C

# What is a Queue?

**Concept**:

- A queue is a linear data structure that follows the **FIFO** (**F**irst **I**n **F**irst **O**ut) rule
- Metaphor: Like a line of people waiting, but in a priority queue, VIPs are served first!

**Basic Operations:**

- **Enqueue**: Add an element at the end (rear) of the queue
- **Dequeue**: Remove an element from the front of the queue
- **Peek / Front**: Return the element at the front without removing it

**Main Types of Queues:**

- **Simple Queue:**
  - Elements are added at the rear and removed from the front (standard FIFO)
  - Used in simple process management
- **Circular Queue:**
  - Rear wraps around to the front
  - Efficient use of space in fixed-size arrays
- **Priority Queue:**
  - Elements are dequeued based on priority, not arrival order
  - Higher-priority elements are served before lower-priority ones

# Queue Operations

## Enqueue

Input: list $Q$, value $x$
Output: list with $x$ added at rear
Instructions:

1. $Q.append(x)$

## Peek / Front

Input: list $Q$
Output: value at the front
Instructions:

1. Return $Q[0]$

## Dequeue

Input: list $Q$
Output: value removed from front
Instructions:

1. $x \leftarrow Q[0]$
2. $Q.remove(x)$
3. Return $x$

# Advantages and Disadvantages of Queues

**Advantages:**

- **Orderly Processing**: Ensures first-come, first-served logic (FIFO)
- **Efficient**: Enqueue and dequeue operations are fast ($\mathcal{O}(1)$)
- **Widely Applicable**:
  - Task scheduling (ex: CPU jobs)
  - Print queues
  - Message queues in communication systems

**Disadvantages:**

- **Limited Access**: Only front element can be read or removed
- **Fixed Capacity in Arrays**: May require resizing or dynamic memory
- **Wasted Space in Array Queues**: Without circular handling, memory may become unusable even when space is available

# Contents

Primitive Data Types

Arrays

Lists

Stacks

Queues

Hash Tables

Structures in C

# Hash Tables

**Definition:**

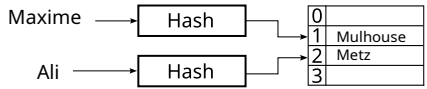
- A hash table stores key–value pairs using a **hash function**
- The hash function computes an index into an array for fast access

**Examples of Key-Value Pairs:**

- `key`: your name → `value`: address
- `key`: your age → `value`: phone number
- `key`: your username → `value`: password

**Simple Hash Function**

```
function hash(key, size):
   sum = 0
   for each char in key:
     sum += ASCII(char)
   return sum % size
```




- Fast lookup, inserts, and deletes, ideally $\mathcal{O}(1)$
- Perfect for mapping keys to values

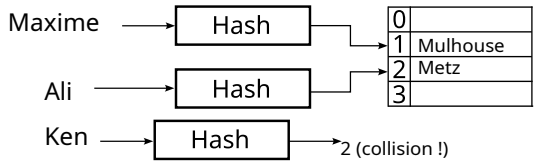
# What is a Collision in Hash Tables?

**Definition:**

- A **collision** occurs when two different keys are hashed to the same index
- Since the index is already occupied, we need a strategy to store the new item

**Example:**

- "Ali" and "Ken" both hash to index 2 → **collision!**




**Collision Resolution Techniques**:

- **Chaining**: Store multiple items at the same index using a list
- **Open Addressing**: Look for the next free slot in the array
- **Rehashing**: Use a second hash function to find another index

# Advantages and Disadvantages of Hash Tables

**Advantages**:

- **Very Fast Access**: Average-case $\mathcal{O}(1)$ for insert, search, and delete
- **Flexible Key Types**: Can use strings, numbers, or other types as keys
- **Efficient for Large Datasets**: Ideal for fast lookups when many elements
- **Widely Used**: In compilers, databases, caches, symbol tables, etc.

**Disadvantages:**

- **Collisions**: Can lead to performance degradation without good resolution strategy
- **Not Ordered**: Elements are not stored in a sorted or predictable order
- **Difficult to Resize**: Resizing requires rehashing all elements
- **Depends on Good Hash Function**: Poor hash functions lead to clustering and collisions

# Contents

Primitive Data Types

Arrays

Lists

Stacks

Queues

Hash Tables

Structures in C

# What is a `struct` in C? Why use it?

- **Definition:** A user-defined type that groups related fields (possibly of different types) under one name
- **Why use it ?**
  - Group related information (example: student data)
  - Facilitate data handling and organization

**Example**

**C Code Example Struct**

```c
#include <stdio.h>

struct Person {
    char name[32];
    int  age;
};
int main(void) {
    struct Person alice = {"Alice", 30};
    printf("%s␣is␣%d\n", alice.name, alice.age);
    return 0;
}
```

# Accessing Members: Direct vs Pointer

**Direct access with .**

## C Code Example Struct

```c
#include <string.h>

struct Person { char name[32]; int age; };
int main(void) {
    struct Person p = {"Ana", 28};
    p.age = 29;                 // dot
    strcpy(p.name, "Ben");      // dot
```

**Indirect access with pointer (->)**

## C Code Example Struct

```c
    struct Person *pp = &p;
    pp->age = 30;               // arrow (preferred)
    (*pp).age = 31;             // equivalent, less readable
    return 0;}
```

# Nested `structs` (Struct containing another Struct)

## C Code Example Struct

```c
#include <stdio.h>

struct Date {
    int year, month, day;
};

struct Person {
    char name[32];
    struct Date dob;     // nested struct
};

int main(void) {
    struct Person p = {"Lina", {1999, 7, 15}};
    printf("%s␣born␣%d-%02d-%02d\n",
           p.name, p.dob.year, p.dob.month, p.dob.day);

    struct Person *pp = &p;
    pp->dob.year = 2000;  // pointer, then nested member
    printf("Updated␣year:␣%d\n", p.dob.year);
    return 0;
}
```

# Pros and Cons of `structs`

**Advantages**

- Groups heterogeneous data logically
- Simplifies the handling of complex data
- Easy to use with arrays and pointers

**Disadvantages**

- No methods/encapsulation; must manage invariants manually
- Must manage lifetime for dynamic members yourself
- No polymorphism/ inheritance; composition only